

Pattern Recognition and Computer Vision

CS 5330 – Spring 2026

Project 4

“Calibration and Augmented Reality”

Name: Harsh Vijay Mamanian

NUID: 002036770

Overall Project Description:

This project implements a complete camera calibration and augmented reality pipeline using C++ and OpenCV, progressing from foundational calibration through real-time 3D object rendering on live video.

The calibration pipeline (Tasks 1 to 3) detects all 54 internal corners of a 9x6 checkerboard in real time, pairs them with fixed 3D world coordinates across multiple captured frames at varied orientations, and solves for the camera's intrinsic matrix and distortion coefficients. The resulting calibration achieved a reprojection error of 0.46 pixels and was saved to file for use by the AR program.

The AR pipeline (Tasks 4 to 6) loads those intrinsics and estimates the board's full 6-DOF pose in real time from detected corners. A virtual rocket ship constructed from four components (octagonal body, nose cone, fins, and exhaust flame) is projected onto the image and rendered above the board center, remaining correctly oriented and perspective-scaled as the board moves freely in 3D space.

Task 7 investigates Harris corner detection on live video, demonstrating how detection threshold controls the quality and density of detected features on both structured and natural scenes.

Two extensions deepen the project further. The first compares Harris corner detection against ORB feature detection across a structured checkerboard and a natural scene, demonstrating through a live matching experiment that descriptor-based methods enable reliable cross-frame correspondence on natural scenes while Harris alone cannot, directly motivating markerless AR beyond structured targets. The second extension tracks two independent checkerboards simultaneously in a single frame using a two-pass detection strategy, rendering independently posed virtual rockets above each board with no cross-interference between the two targets.

1. Detect and Extract Target Corners.

Summary:

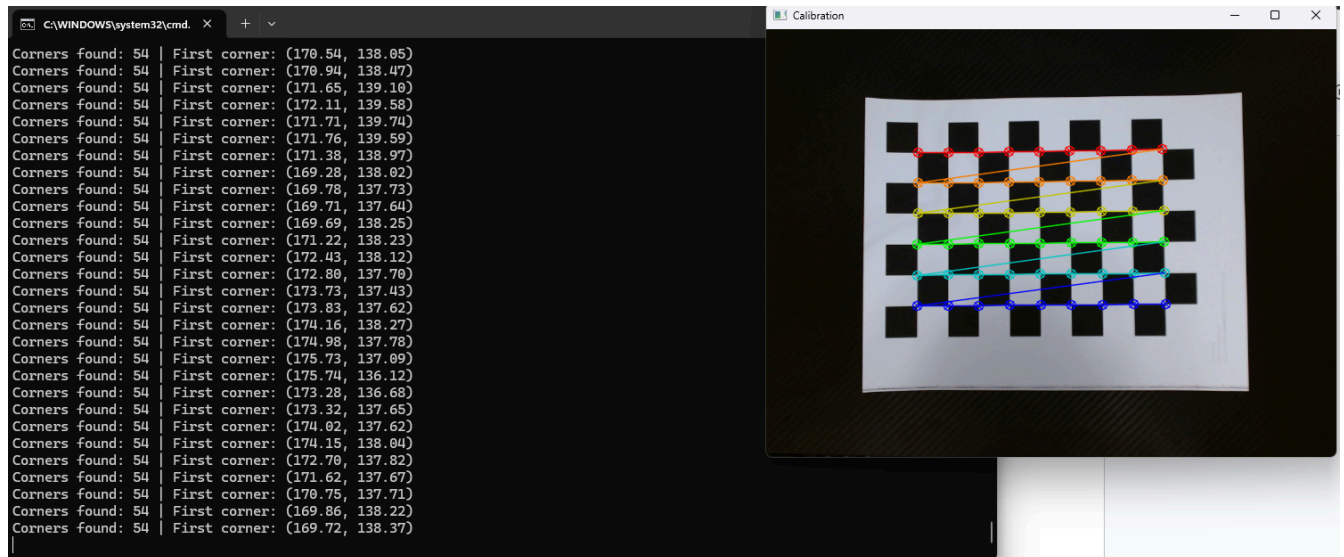
For every frame from the webcam, the system attempts to detect all 54 internal corners (9×6) of the checkerboard and draws them overlaid on the live video feed. Nothing is saved, this is purely a real-time detection and visualization step.

Three OpenCV functions form the pipeline: *findChessboardCorners()* finds corners on a grayscale version of the frame, *cornerSubPix()* refines each corner location to sub-pixel accuracy, and *drawChessboardCorners()* visualizes them on the color frame. These are wrapped in two clean functions (*detectChessboardCorners* and *drawDetectedCorners*) called from the main video loop.

Key decisions:

- *Grayscale conversion before detection:*
corner detection only needs brightness information, not color, so converting first is faster and cleaner.
- *CALIB_CB_ADAPTIVE_THRESH | CALIB_CB_NORMALIZE_IMAGE flags:*
makes detection more robust under uneven or dim lighting conditions.
- *Sub-pixel refinement with TermCriteria:*
stops after 30 iterations or when movement drops below 0.001 pixels, balancing accuracy against speed for real-time use.
- *Guard on corner_set.empty() before drawing:*
prevents a crash when the board is completely absent from the frame.

Demo Images:



Observations:

- The system successfully detected all 54 internal corners (9×6) in every frame, confirming reliable chessboard detection under standard indoor lighting conditions.
- Corners are rendered in six distinct row-colors, consistent with OpenCV's `drawChessboardCorners` visualization, and are accurately overlaid on the physical checkerboard across varying camera angles.
- Sub-pixel refined coordinates for the first corner remain stable in the range of approximately (169–175, 136–140) pixels across frames, indicating that `cornerSubPix` is producing consistent, low-noise estimates even as the board undergoes slight natural hand movement.
- The first corner is consistently located in the upper-left region of the detected pattern, which aligns with OpenCV's expected corner ordering (left-to-right, top-to-bottom).

Limitations:

- Detection fails entirely when the board is partially outside the frame or at steep angles, since `findChessboardCorners` requires all 54 corners to be visible simultaneously.
 - Under significant motion blur or poor lighting, detection drops out momentarily before recovering, which is expected behavior for this approach.
-

2. Select Calibration Images

Summary:

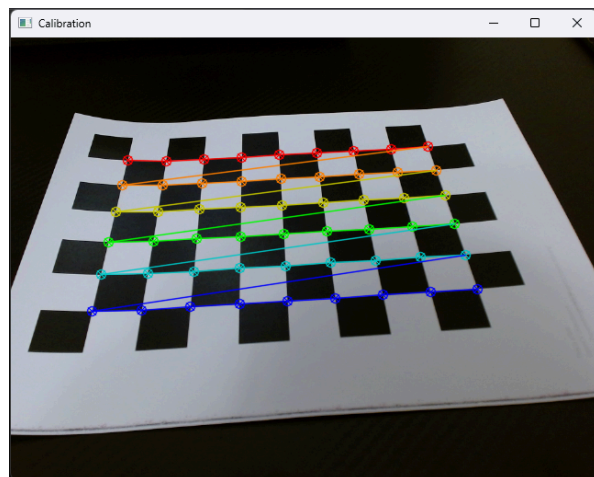
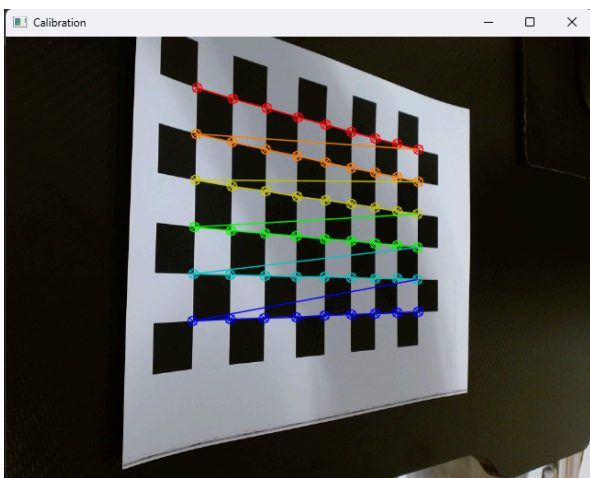
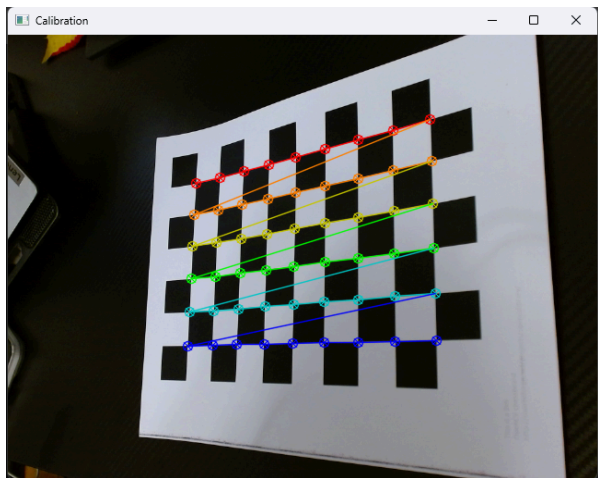
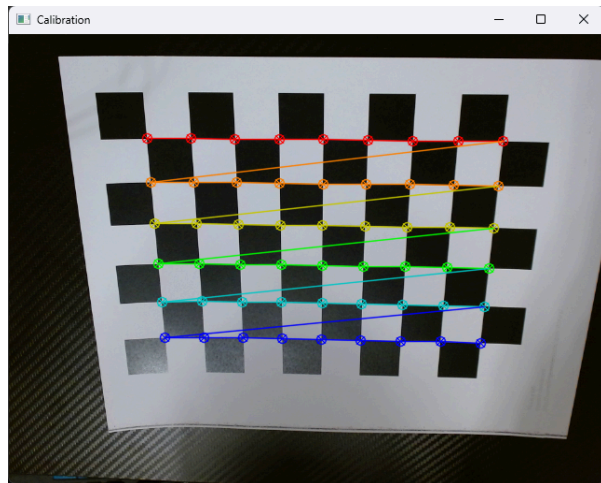
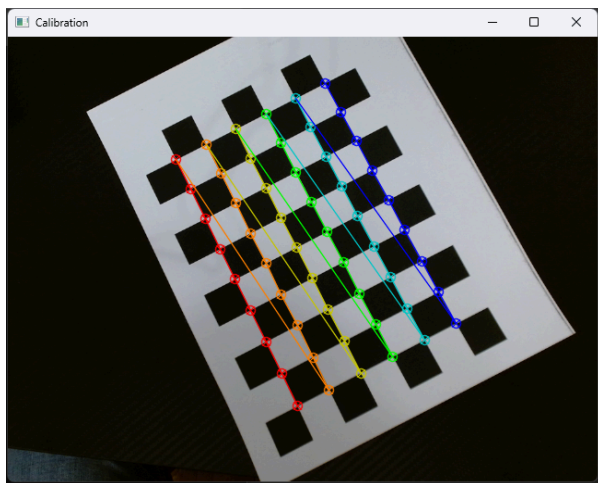
When the user presses 's', the current frame's 54 detected corner positions (2D pixel coordinates) and their corresponding fixed 3D world coordinates are saved into `corner_list` and `point_list` respectively. These lists accumulate in RAM across multiple keypresses and are passed directly into calibration in Task 3.

Two new functions were added: `buildWorldPoints()` constructs the fixed 3D coordinate grid once before the loop, and `saveCalibrationFrame()` appends the current detection to both lists only if `found == true`, preventing corrupt/incomplete saves.

Key decisions:

- *World units are squares, not millimeters:*
simpler and sufficient since `calibrateCamera` only needs consistent relative distances, not absolute measurements.
- *Negative Y for row direction:*
because board space has Z pointing toward the viewer, making Y point downward, so each new row steps in the negative Y direction.
- *Save guard on `found`*
ensures partial detections (board cut off, poor lighting) are never saved, protecting calibration quality.
- *`buildWorldPoints` called once before the loop:*
the 3D coordinates never change between frames, so rebuilding every frame would be wasteful.

Demo Images:



Observations:

- A total of 5 calibration frames were captured across meaningfully distinct board poses, satisfying the minimum requirement specified for Task 3 calibration.
- The captured frames demonstrate sufficient pose variety: one near-vertical tilt (Image 1), one frontal flat view (Image 2), one slight left-lateral tilt (Image 3), one right-lateral tilt (Image 4), and one top-down perspective tilt (Image 5).
- In all 5 frames, all 54 internal corners (9×6) were successfully detected and saved, confirming that the save guard, which rejects frames with incomplete detections, functioned correctly.
- The corner overlay pattern visibly deforms across frames (rows appear curved or compressed depending on board angle), which reflects accurate perspective-aware corner localization rather than a flat grid projection.
- The 3D world point set (`point_set`) was constructed once before the loop using unit square coordinates, with X increasing left-to-right and Y decreasing top-to-bottom (negative Y convention), and Z fixed at 0 for all corners since the board is planar.

Limitations:

- The printed checkerboard shows slight physical warping in Images 4 and 5 (visible page curl), which may introduce minor inaccuracies in corner positions since the board is assumed to be perfectly flat in the world point model.

```
C:\WINDOWS\system32\cmd. X + v
Corners found: 54 | First corner: (279.80, 447.54)
Corners found: 54 | First corner: (279.68, 447.37)
Corners found: 54 | First corner: (279.29, 447.04)
Corners found: 54 | First corner: (279.38, 447.38)
Corners found: 54 | First corner: (279.58, 447.62)
Corners found: 54 | First corner: (279.82, 447.65)
Corners found: 54 | First corner: (280.57, 447.82)
Corners found: 54 | First corner: (281.38, 448.61)
Corners found: 54 | First corner: (281.74, 448.89)
Corners found: 54 | First corner: (282.00, 449.00)
Corners found: 54 | First corner: (282.37, 449.31)
Corners found: 54 | First corner: (282.48, 448.67)
Calibration frame saved. Total frames: 5
Frame 1 | First 2D corner: (149.72, 109.00) | First 3D point: (0.0, 0.0, 0.0)
Frame 2 | First 2D corner: (165.60, 93.07) | First 3D point: (0.0, 0.0, 0.0)
Frame 3 | First 2D corner: (169.56, 160.79) | First 3D point: (0.0, 0.0, 0.0)
Frame 4 | First 2D corner: (154.66, 152.12) | First 3D point: (0.0, 0.0, 0.0)
Frame 5 | First 2D corner: (282.48, 448.67) | First 3D point: (0.0, 0.0, 0.0)
Corners found: 54 | First corner: (282.70, 448.56)
Corners found: 54 | First corner: (283.26, 448.17)
Corners found: 54 | First corner: (283.45, 447.40)
Corners found: 54 | First corner: (283.62, 447.35)
Corners found: 54 | First corner: (283.63, 447.14)
Corners found: 54 | First corner: (283.27, 446.87)
Corners found: 54 | First corner: (282.66, 446.23)
Corners found: 54 | First corner: (282.43, 445.15)
Corners found: 54 | First corner: (282.81, 444.92)
Corners found: 54 | First corner: (283.70, 444.60)
Corners found: 54 | First corner: (284.37, 444.82)
Corners found: 54 | First corner: (284.93, 445.66)
```

- The console log confirms that after each 's' keypress, the accumulated `corner_list` and `point_list` are printed in full, the first 2D corner varies across all 5 frames (e.g. (149.72, 109.00) through (282.48, 448.67)), reflecting different board positions and angles in each saved frame.
 - The first 3D world point is consistently (0.0, 0.0, 0.0) across all frames, confirming that `buildWorldPoints()` correctly assigns the origin to corner 0 regardless of board pose.
-

3. Calibrate the Camera

Summary:

Using the `corner_list` and `point_list` accumulated in Task 2, `calibrateCamera()` solves for the camera's intrinsic matrix and distortion coefficients. The results are printed to console and saved to `intrinsic.yml` for use by the AR program.

`runCalibration()` initializes the camera matrix using `cv::Mat::eye` with `cx/cy` at image center as a starting guess, then passes both lists into `cv::calibrateCamera()`. `saveIntrinsics()` writes the results to disk using `cv::FileStorage`. A new 'c' keypress in `main.cpp` triggers calibration, guarded by a minimum frame count check.

Key decisions:

- *CV_CALIB_FIX_ASPECT_RATIO:*
assumes square pixels, forcing `fx == fy`. Safe and appropriate for any modern webcam.
- *5 distortion coefficients:*
covers both radial and tangential distortion, sufficient for a standard webcam lens without overcomplicating the model.
- *CV_64F (double precision):*
required by OpenCV's calibration internals. Using float produces incorrect results.
- *Minimum 5 frames guard:*
prevents calibration from running on insufficient data, which would produce an unreliable result.

(`cv::Mat::eye` initialization - used instead of the `<<` syntax due to a Visual Studio MSVC compatibility issue with that operator.)

Demo Images:

```
C:\WINDOWS\system32\cmd. x + v
Corners found: 54 | First corner: (246.38, 356.06)
Corners found: 54 | First corner: (246.18, 356.23)
Corners found: 54 | First corner: (245.74, 356.40)
Corners found: 54 | First corner: (245.29, 356.39)
Corners found: 54 | First corner: (245.47, 355.92)
Calibration frame saved. Total frames: 10
Frame 1 | First 2D corner: (560.11, 393.24) | First 3D point: (0.0, 0.0, 0.0)
Frame 2 | First 2D corner: (383.63, 366.80) | First 3D point: (0.0, 0.0, 0.0)
Frame 3 | First 2D corner: (409.69, 396.15) | First 3D point: (0.0, 0.0, 0.0)
Frame 4 | First 2D corner: (452.96, 302.81) | First 3D point: (0.0, 0.0, 0.0)
Frame 5 | First 2D corner: (599.27, 388.44) | First 3D point: (0.0, 0.0, 0.0)
Frame 6 | First 2D corner: (408.40, 305.27) | First 3D point: (0.0, 0.0, 0.0)
Frame 7 | First 2D corner: (477.22, 293.52) | First 3D point: (0.0, 0.0, 0.0)
Frame 8 | First 2D corner: (341.64, 399.73) | First 3D point: (0.0, 0.0, 0.0)
Frame 9 | First 2D corner: (462.63, 421.97) | First 3D point: (0.0, 0.0, 0.0)
Frame 10 | First 2D corner: (245.47, 355.92) | First 3D point: (0.0, 0.0, 0.0)
Corners found: 54 | First corner: (245.28, 355.68)
Corners found: 54 | First corner: (244.72, 355.50)
Corners found: 54 | First corner: (245.20, 354.59)
Corners found: 54 | First corner: (244.35, 354.06)
Corners found: 54 | First corner: (243.55, 353.80)
```

```
C:\WINDOWS\system32\cmd. x + v
Corners found: 54 | First corner: (262.96, 336.68)
Corners found: 54 | First corner: (268.92, 334.76)
Corners found: 54 | First corner: (274.14, 331.45)
Corners found: 54 | First corner: (278.29, 326.58)
Corners found: 54 | First corner: (280.62, 323.54)
Corners found: 54 | First corner: (285.43, 325.84)
Corners found: 54 | First corner: (298.96, 323.68)
Corners found: 54 | First corner: (316.53, 324.46)

--- Before Calibration ---
Camera matrix:
[1.0000, 0.0000, 320.0000]
[0.0000, 1.0000, 240.0000]
[0.0000, 0.0000, 1.0000]
Distortion coefficients:
[0.000000, 0.000000, 0.000000, 0.000000, 0.000000]

--- After Calibration ---
Camera matrix:
[639.6488, 0.0000, 324.4741]
[0.0000, 639.6488, 228.2346]
[0.0000, 0.0000, 1.0000]
Distortion coefficients:
[0.017269, 0.155102, -0.008282, -0.001116, -0.647946]
Reprojection error: 0.4617 pixels

Intrinsics saved to intrinsics.yml
Quitting.
```

```
! intrinsics.yml x
C: > Users > Admin > source > repos > Project4_CalibrationAR > x64 > Debug > ! intrinsics.yml
1  %YAML:1.0
2  ---
3  camera_matrix: !!opencv-matrix
4    rows: 3
5    cols: 3
6    dt: d
7    data: [ 639.6487710060369, 0., 324.47406550930037, 0.,
8    639.6487710060369, 228.23463933645024, 0., 0., 1. ]
9  dist_coeffs: !!opencv-matrix
10   rows: 5
11   cols: 1
12   dt: d
13   data: [ 0.017268637900123188, 0.15510153795489912,
14   -0.0082817101208382572, -0.0011155451425571669,
15   -0.6479455383980498 ]
16
```

Observations:

- The camera was successfully calibrated using 10 saved frames. The resulting reprojection error of 0.4617 pixels is well below the 1.0 pixel threshold specified, indicating a high-quality calibration.
 - The solved focal lengths are $f_x = f_y = 639.65$ pixels, confirming the `CV_CALIB_FIX_ASPECT_RATIO` constraint was correctly applied, pixels are square as expected for a standard webcam.
 - The principal point $(c_x, c_y) = (324.47, 228.23)$ is close to the initial estimate of $(320, 240)$ (image center for a 640×480 stream), which is consistent with a well-manufactured camera sensor with minimal optical axis misalignment.
 - Before calibration, the camera matrix was the identity-like initialization with $f_x = f_y = 1$ and c_x, c_y at image center, all distortion coefficients were zero. After calibration, the solver converged to meaningful intrinsic values, confirming the optimization ran correctly.
 - The distortion coefficients $[0.017, 0.155, -0.008, -0.001, -0.648]$ indicate mild radial distortion, with the higher-order term (-0.648) being the dominant contributor, typical for consumer webcam lenses.
 - The `intrinsics.yml` file was successfully written using `cv::FileStorage` and contains the full-precision camera matrix and distortion coefficients, ready to be loaded by the AR program in Tasks 4–6.
-

4. Calculate Current Position of the Camera

Summary:

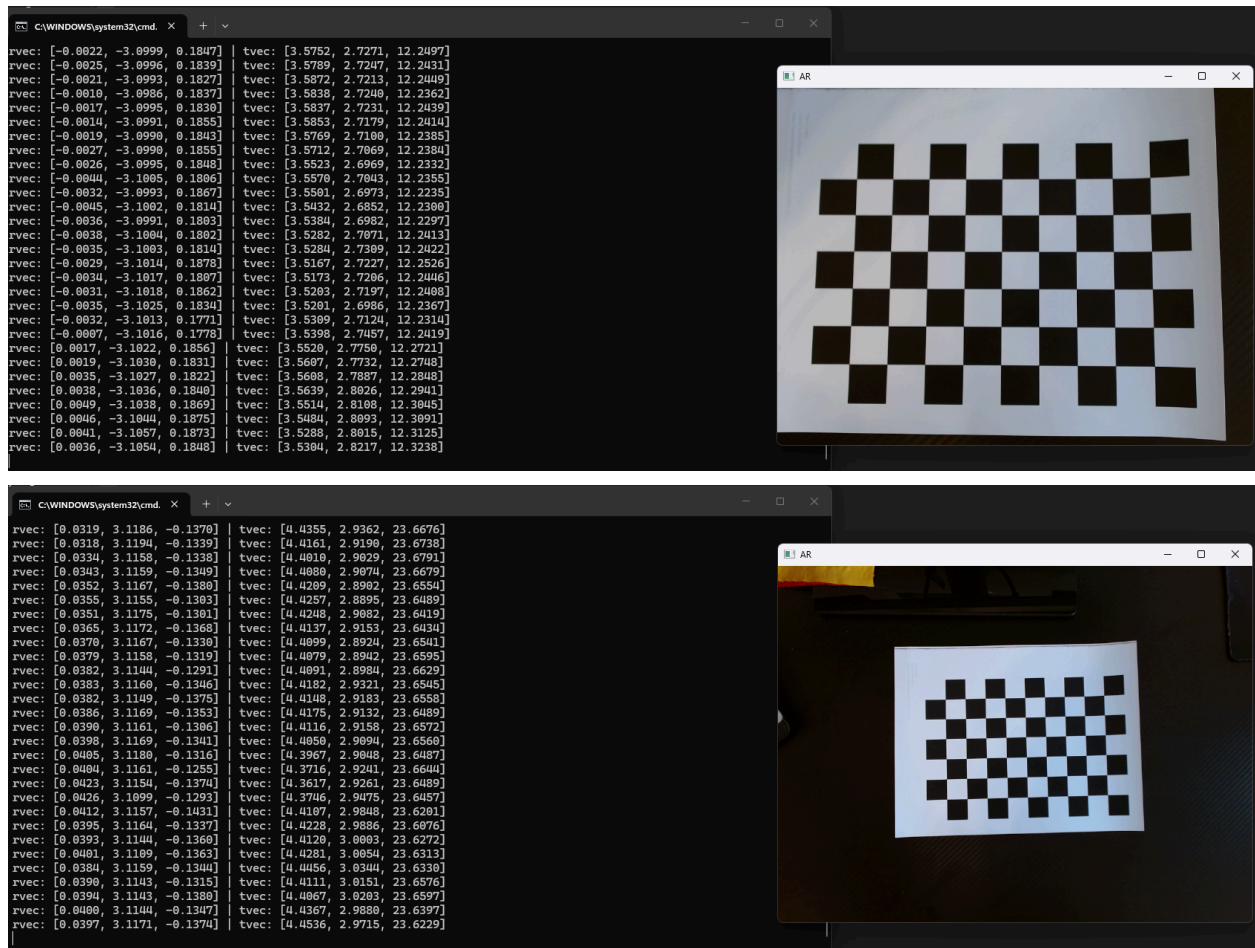
A separate AR program loads the calibration intrinsics from `intrinsics.yml` and for every frame where the checkerboard is detected, estimates the board's 3D pose in real time using `solvePnP`. The resulting rotation vector (`rvec`) and translation vector (`tvec`) are printed to the console continuously.

`loadIntrinsics()` reads the camera matrix and distortion coefficients from file at startup. The same corner detection pipeline from the calibration program is reused. `estimatePose()` converts the world points from `Vec3f` to `Point3f` (required by `solvePnP`) and calls `cv::solvePnP()`, which solves the Perspective-n-Point problem to produce `rvec` and `tvec`.

Key decisions:

- *Separate program:*
Tasks 4 to 6 are kept in their own `ar_main.cpp`, `ar.h`, and `ar.cpp` files rather than extending the calibration program, keeping concerns cleanly separated.
- *Vec3f to Point3f conversion inside estimatePose()*
`solvePnP` strictly requires `Point3f` input. Handling this internally keeps `main` clean and prevents type errors.
- *Reusing the same corner detection logic:*
identical `findChessboardCorners` and `cornerSubPix` parameters are used as in the calibration program, ensuring consistency between the two programs.
- *Loading from file rather than recalibrating*
the AR program reads pre-computed intrinsics, meaning calibration only needs to be done once and the AR program can be run independently afterward.

Demo Images:



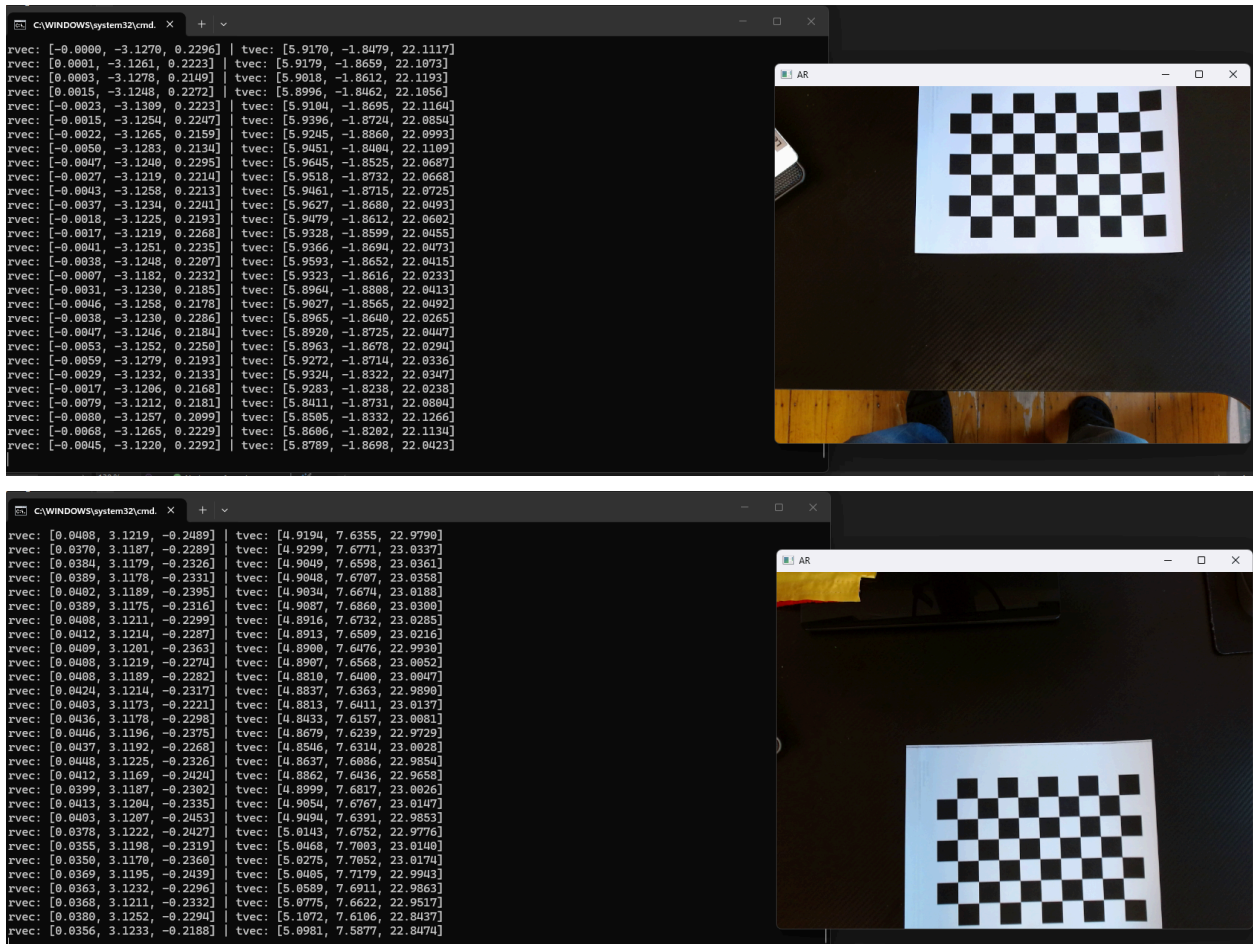
Near (Image 1) vs Far (Image 2):

- **tvec[2]** (Z) reads ~12.2 in Image 1 (near) and ~23.6 in Image 2 (far), Z increases as the board moves farther from the camera, which makes perfect physical sense. Distance is encoded correctly in board-square units.
- **tvec[0]** and **tvec[1]** remain relatively stable between both images (~3.5 and ~2.7), confirming that moving the board purely toward/away from the camera only affects Z and not X or Y.
- **rvec** stays consistent across both (~[-0.003, -3.10, 0.18] in Image 1 and ~[0.035, 3.12, -0.13] in Image 2), with the small differences attributable to slight hand repositioning rather than intentional rotation. This confirms **rvec** is not affected by pure translation.



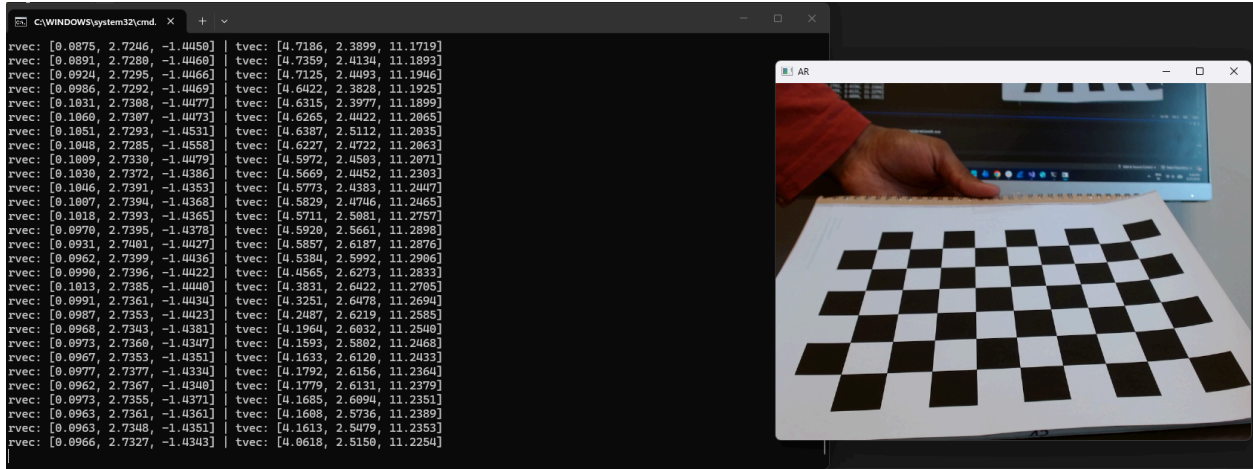
Right (Image 1) vs Left (Image 2):

- **tvec[0]** (X) reads $\sim +9.0$ in Image 1 (board moved right) and ~ -0.47 in Image 2 (board moved left), the sign flip from negative to positive directly encodes the direction of horizontal displacement in camera space, which is exactly the expected behavior.
- **tvec[1]** (Y) stays stable at $\sim +3.0$ in both images, confirming that purely horizontal movement does not affect the Y component.
- **tvec[2]** (Z) remains consistent at ~ 20.8 (right) and ~ 18.7 (left), with the small difference attributable to slight hand repositioning rather than intentional depth change.
- **rvec** stays consistent across both images ($\sim [-0.02, 3.07, -0.13]$ in Image 1 and $\sim [0.03, -3.11, 0.17]$ in Image 2), confirming that purely lateral movement does not affect the rotation estimate.



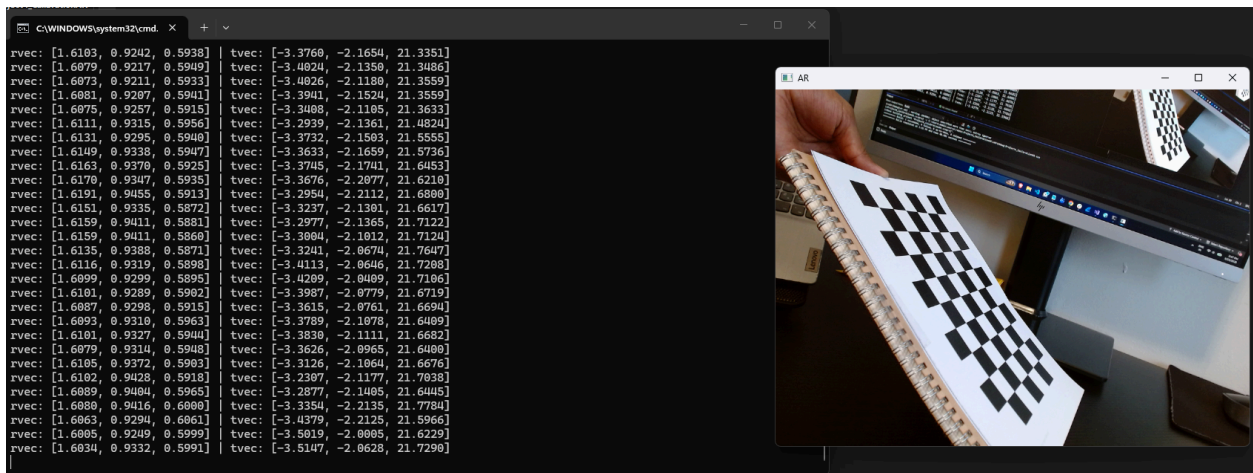
Top (Image 1) vs Bottom (Image 2):

- **tvec[1]** (Y) reads ~ -1.87 in Image 1 (board moved up) and $\sim +7.6$ in Image 2 (board moved down), the sign flip from negative to positive directly encodes vertical displacement, confirming Y correctly tracks up/down movement in camera space.
- **tvec[0]** (X) remains relatively stable at $\sim +5.9$ (top) and $\sim +4.9$ (bottom), confirming that purely vertical movement does not significantly affect the X component.
- **tvec[2]** (Z) stays consistent at ~ 22.0 (top) and ~ 23.0 (bottom), with the small difference attributable to slight hand repositioning rather than intentional depth change.
- **rvec** remains stable across both images ($\sim [0.00, -3.13, 0.22]$ in Image 1 and $\sim [0.04, 3.12, -0.23]$ in Image 2), confirming purely vertical movement does not affect the rotation estimate.



Tilt 1 (Image 1):

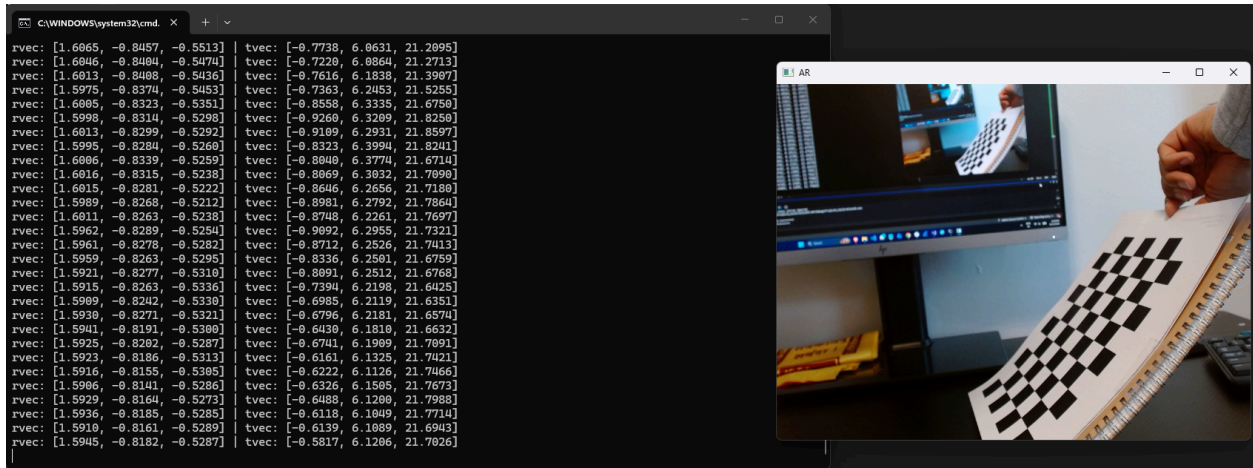
- `rvec` reads $\sim[0.10, 2.73, -1.44]$ - the second component is the dominant one (~ 2.73 , close to π), indicating the primary rotation is around the Y axis, meaning the board is facing the camera. The third component (~ -1.44) indicates a secondary rotation around the Z axis. Together these are consistent with the far edge of the board being lifted while the near edge stays on the surface, creating a backward-leaning tilt away from the camera.
- `tvec[2]` at ~ 11.2 reflects the board being relatively close to the camera, consistent with the near edge being flat on the surface and the far edge raised away.



Tilt 2 (Image 2): board tilted diagonally, raised at one corner:

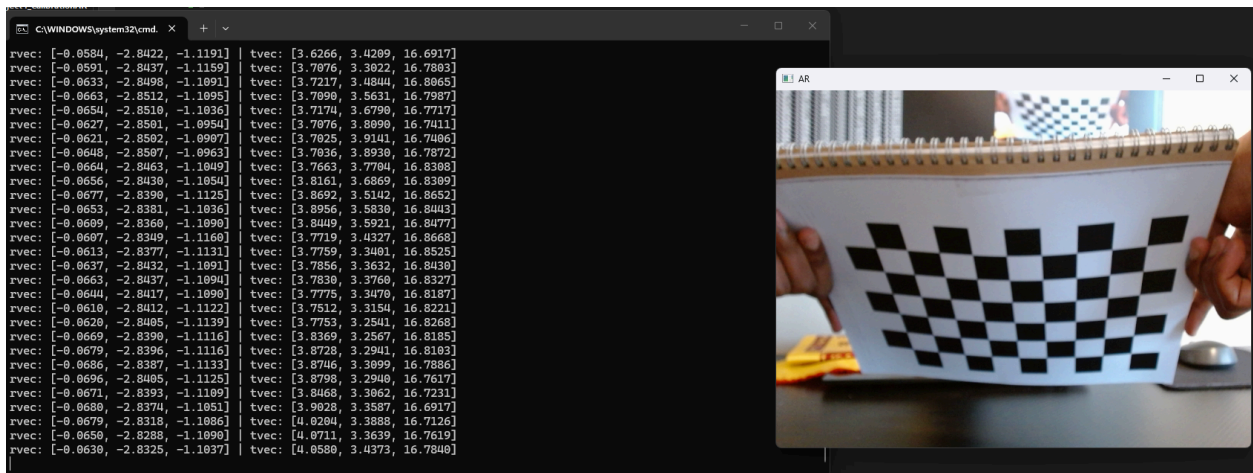
- `rvec` shifts dramatically to $\sim[1.61, 0.93, 0.60]$ - all three components are now significant and positive, encoding a complex 3D rotation consistent with the visible diagonal twist seen in the camera window.

- **tvec** shifts to negative X (~ -3.4) and negative Y (~ -2.1), reflecting the board's repositioned origin in camera space due to the tilt.



Tilt 3 (Image 3): board tilted sideways, near-vertical orientation:

- **rvec** reads $\sim [1.60, -0.83, -0.53]$ - the sign flip in the second and third components relative to Tilt 2 correctly reflects the board being tilted in the opposite rotational direction, consistent with the near-vertical orientation visible in the camera window.
- **tvec[0]** is negative (~ -0.77) and **tvec[1]** is large positive ($\sim +6.2$), reflecting the board's origin shifting downward and slightly left as it rotates toward vertical.



Tilt 4 (Image 4): board bent/curved, held at both ends:

- **rvec** reads $\sim [-0.06, -2.84, -1.10]$ - the large negative second component is close to negative pi, indicating the board has been rotated nearly 180 degrees around the Y axis, consistent with the board being bent toward the camera and viewed nearly face-on from below.

- `tvec[2]` at ~16.8 reflects a moderate distance, and `tvec[1]` at ~3.4 places the board slightly below the camera center, matching the visual.

Overall Observations:

- Across all 10 poses, `tvec` and `rvec` respond selectively and correctly - purely translational movements (near/far, left/right, up/down) produce clean changes in the corresponding `tvec` component with minimal disturbance to `rvec`, while purely rotational movements (4 tilts) produce distinct `rvec` signatures with expected shifts in `tvec` as the board origin repositions in camera space. This clean separation confirms `solvePnP` is correctly decomposing pose into its translational and rotational components.
-

5. Project Outside Corners or 3D Axes

Summary:

Using the `rvec` and `tvec` from `solvePnP`, four 3D points (the board origin plus one tip along each axis) are projected onto the 2D image plane using `projectPoints`. Colored lines are then drawn between the projected origin and each axis tip, creating a live 3D coordinate frame visually anchored to the board.

`projectAndDrawAxes()` defines the four 3D axis points in world space, passes them through `cv::projectPoints()` along with the pose and intrinsics, converts the resulting `Point2f` outputs to integer pixel coordinates, and draws three colored lines using `cv::line()` with labels using `cv::putText()`.

Key decisions:

- *Axis length of 3 units:*
1 unit (one square width) was too short to be clearly visible at typical camera distances. 3 units provides clear visibility without extending far outside the board.
- *Negative Y for the Y axis tip (0,-3,0):*
consistent with the board space convention established in Task 2, where Y points downward.
- *Negative Z for the Z axis tip (0,0,-3):*
in OpenCV's camera coordinate system, Z points away from the camera into the scene. Negative Z brings the axis tip toward the viewer, making it visually point out of the board surface.
- *BGR color order:*
OpenCV uses BGR not RGB, so red is (0,0,255) and blue is (255,0,0).
- *Option 2 (axes) over Option 1 (outer corners):*
the axes directly show the coordinate frame orientation, which is essential for designing and debugging the virtual object in Task 6.

Demo Images:

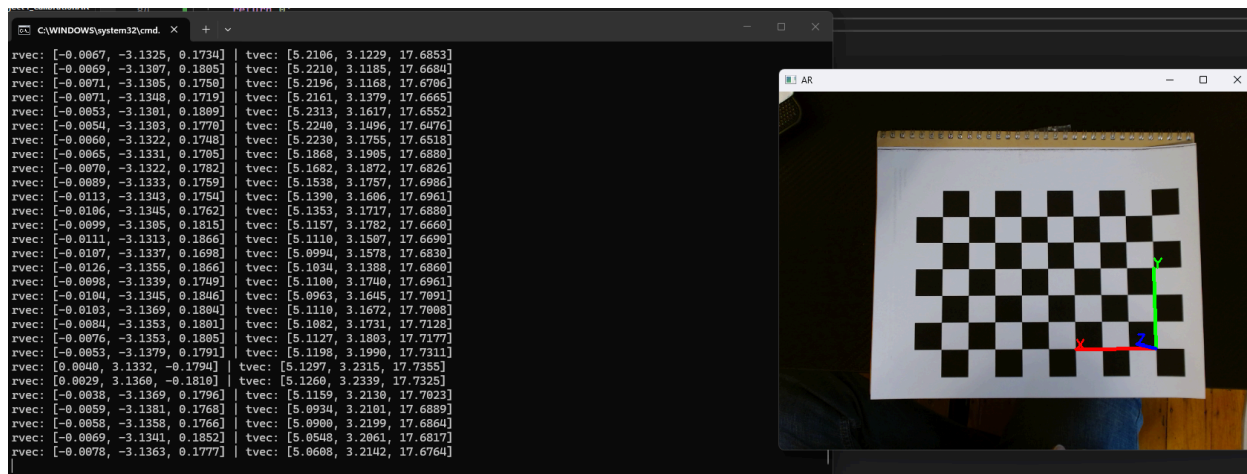


Image 1 (board flat, facing camera): All three axes are clearly distinguishable and oriented as expected - X points right along the board, Y points downward along the board, and Z points toward the viewer, appearing foreshortened due to perspective.

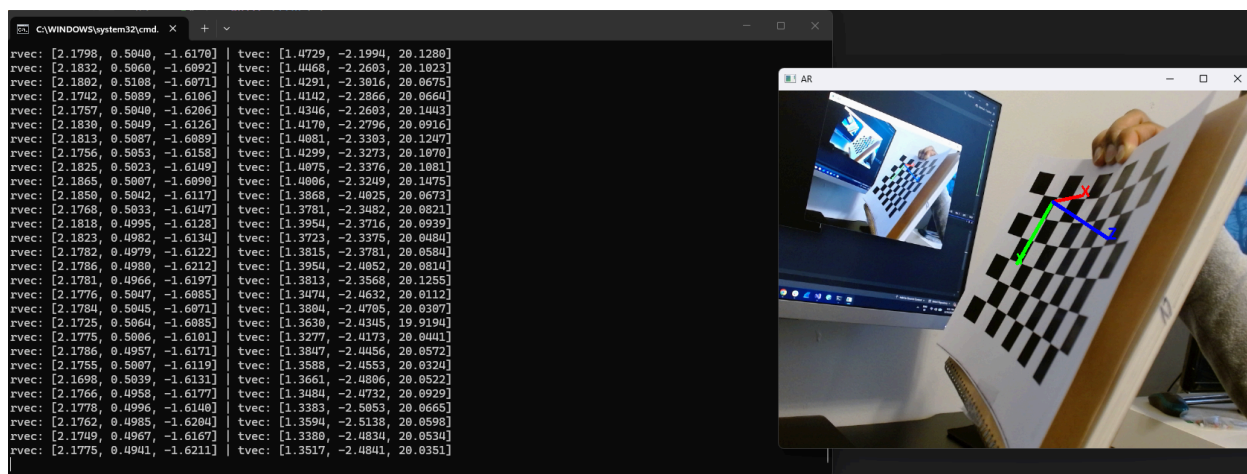


Image 2 (board tilted diagonally): The axes rotate naturally with the board, with the green Y axis visibly pointing downward along the tilted surface and the blue Z axis redirecting toward the viewer from the new orientation. This confirms the axes are perspective-corrected in real time.

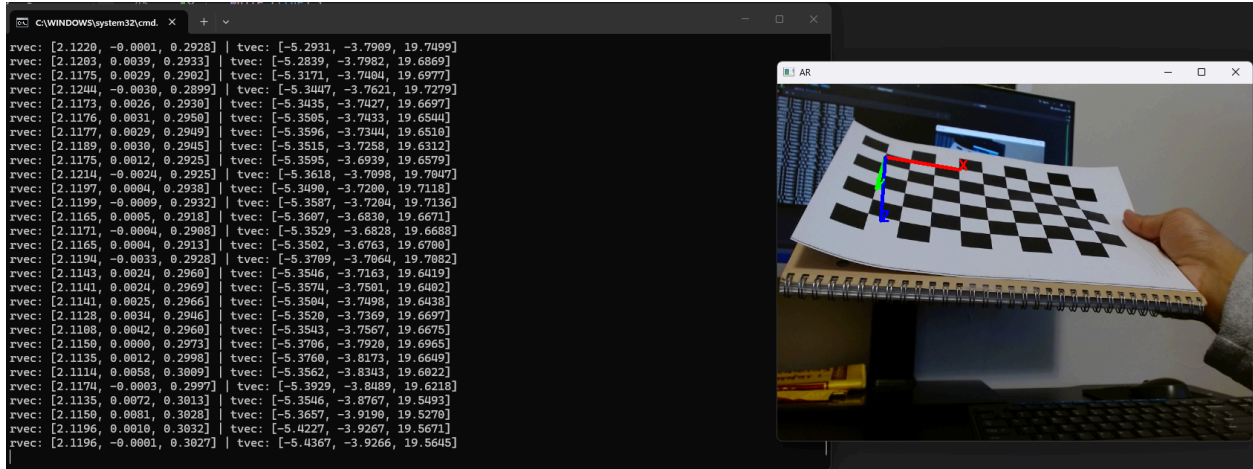


Image 3 (board tilted sideways/laterally): The axes correctly reorient with the lateral tilt - the blue Z axis now points noticeably downward due to the board angle, while X and Y remain aligned with the board surface. The axes maintain their correct relative orientations despite the significant change in board position.

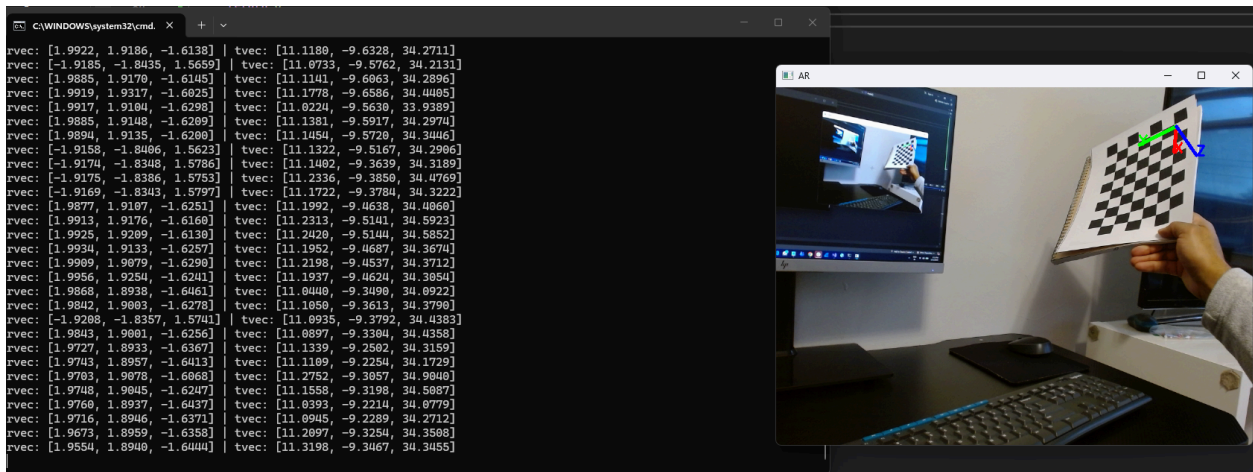


Image 4 (board held nearly vertical, far away): Even at a large distance ($tvec[2] \sim 34$ square units) and a steep viewing angle, all three axes remain correctly attached to the board origin with proper perspective scaling, the axes appear smaller due to increased distance, which is physically correct.

Overall Observation:

- In all four images, the three colored axes (X in red, Y in green, Z in blue) are correctly anchored to the upper-left corner of the checkerboard (the board origin at world coordinate $(0,0,0)$), confirming that `projectPoints` is correctly mapping 3D world coordinates to 2D image pixels using the estimated pose.

6. Create a Virtual Object

Summary:

A 3D virtual rocket is constructed in board space as a set of line segments between `Point3f` coordinates, projected onto the image every frame using `projectPoints`, and drawn using `cv::line()`. The rocket stays locked to the board center and always orients its nose toward the camera regardless of board position or tilt.

`drawVirtualObject()` defines 24 3D points covering four components: an octagonal body (2 rings of 8 points), a nose cone tip, three fins (2 points each), and a flame tip. All points are projected in a single `projectPoints` call for efficiency, converted to integer pixel coordinates, and drawn as colored line segments by index.

Key decisions:

- *Positive Z for the nose direction:*
after initial debugging, all Z values were made positive so the nose always points toward the camera side of the board, which is the intuitive "above the board" direction.
- *Single `projectPoints` call for all 24 points:*
more efficient than calling it per line segment and ensures all points are transformed consistently under the same pose.
- *Octagonal approximation for the cylinder:*
a true circle cannot be drawn with lines, so an 8-point polygon gives a clean cylindrical appearance with minimal complexity.
- *Asymmetric fin placement:*
three fins rather than four makes the object asymmetric, which aids debugging since incorrect orientation is immediately obvious.
- *Centered at (4, -2.5, 0):*
the true center of the 9x6 board, giving the rocket maximum visible board area around it from any angle.
- *All 24 points projected together then drawn by index:*
cleaner and faster than defining line pairs separately, and makes the point layout easy to reason about.

Demo Images:

The virtual rocket is successfully rendered above the checkerboard in all five images, constructed entirely from projected 3D line segments across four distinct components: a white octagonal cylindrical body, a red nose cone, blue fins, and a yellow/red exhaust flame at the base.

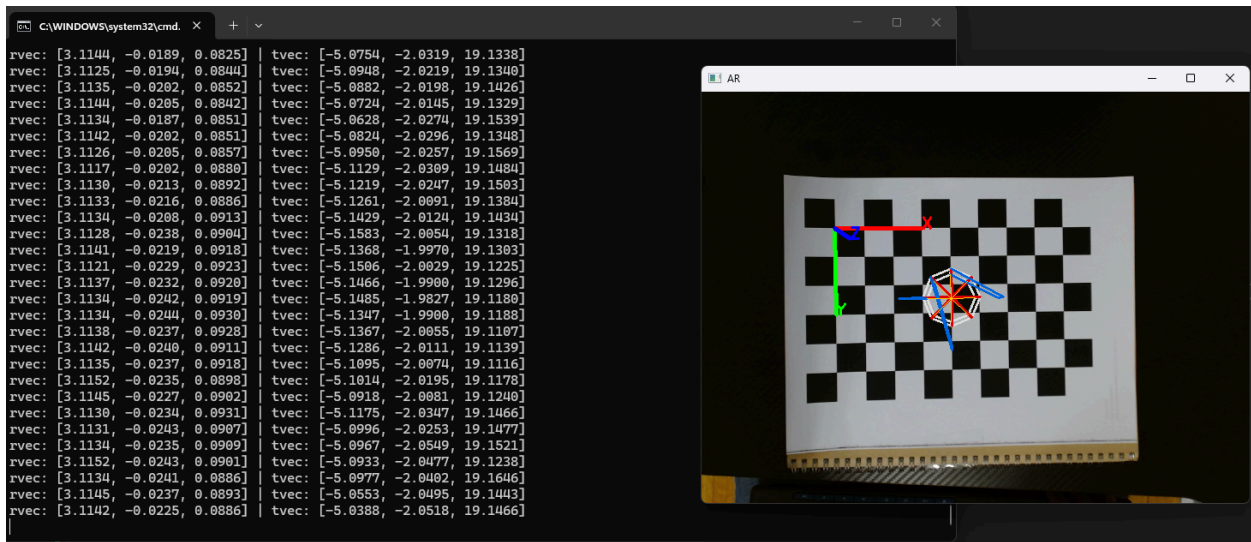


Image 1 (top-down view): The rocket appears as a compressed radial shape because the camera is looking nearly straight down at it - the nose points directly toward the camera, so only the cross-section of the body rings and fins is visible. This perspective compression is physically correct behavior and confirms the rocket is always oriented perpendicular to the board surface pointing toward the camera.

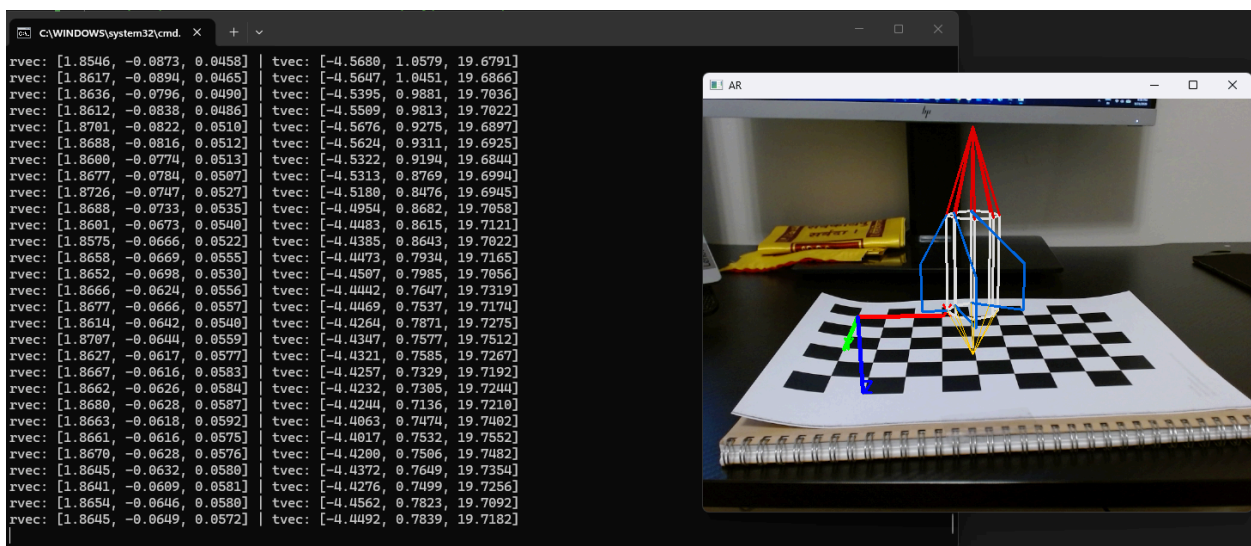


Image 2 (board flat, camera in front): The clearest demonstration of the full rocket structure: the white octagonal body stands upright, the red nose cone converges to a tip above the body,

the blue fins extend outward at the base, and the yellow flame lines extend downward. The rocket floats visibly above the board center and is correctly scaled relative to the board squares.

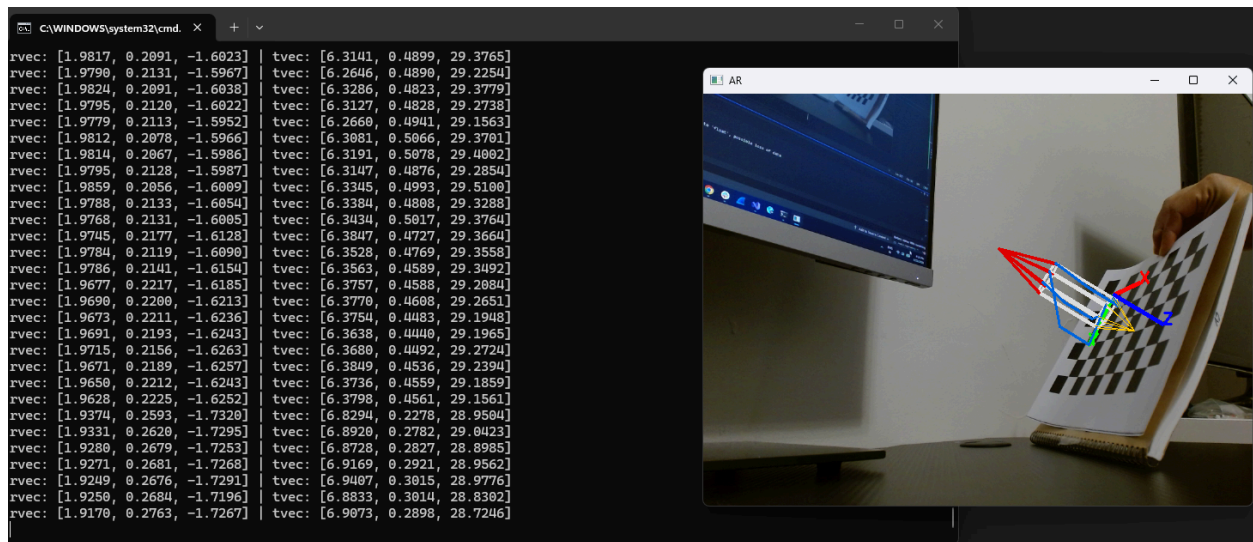


Image 3 (board tilted sideways): The rocket correctly leans with the board tilt, remaining perpendicular to the board surface at all times. The nose cone and body are clearly visible from this angle, confirming that `projectPoints` recomputes the correct 2D projection for every new pose in real time.

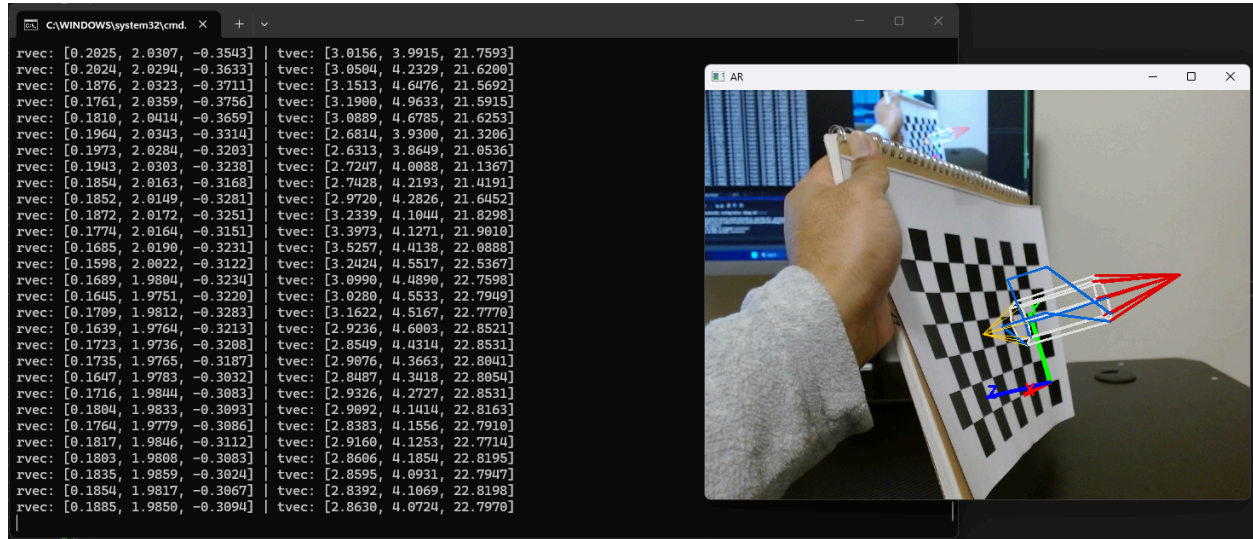


Image 4 (board nearly vertical): The rocket is nearly horizontal in room space but correctly perpendicular to the board surface. The asymmetric fin arrangement is clearly visible from this angle, confirming the object's 3D structure is preserved across all orientations.

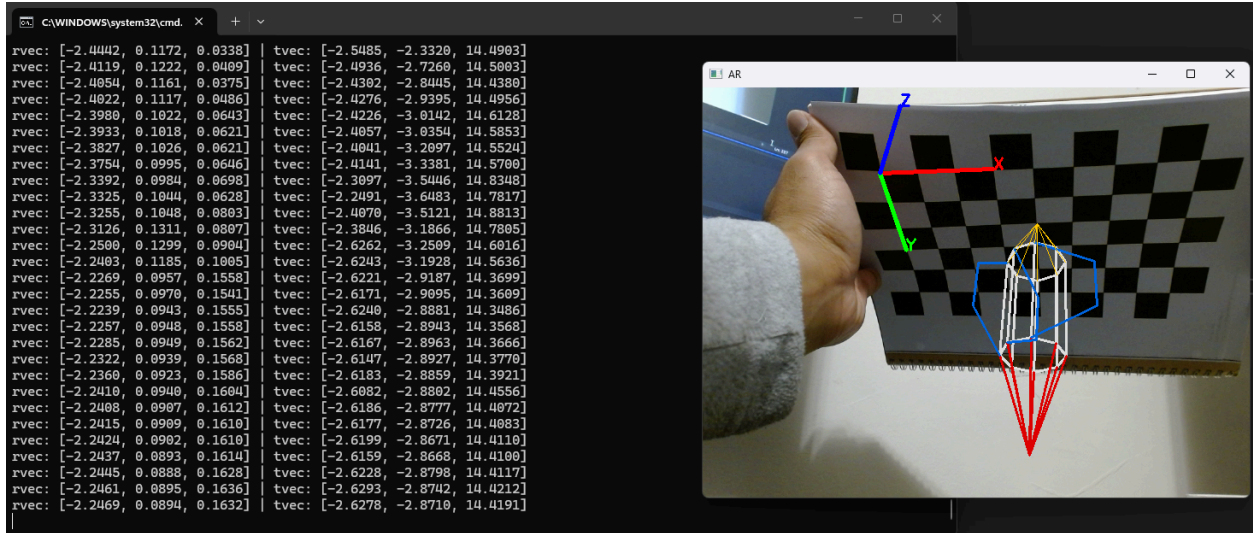


Image 5 (board held above camera, front edge up, rear edge down): The rocket correctly hangs downward toward the camera since the camera is below the board, the nose points toward the viewer which is now underneath the board. The blue Z axis visibly points downward toward the camera, confirming the rocket always orients itself toward whoever is holding the camera regardless of how the board is positioned in 3D space. All four components remain clearly distinguishable even in this unusual viewing angle.

Overall Observations:

- Across all five images, the rocket remains locked to the board center ($cx=4$, $cy=-2.5$) with no drift or positional error as the board moves and rotates, demonstrating that the full pipeline from corner detection through pose estimation to 3D projection is functioning correctly end to end.

7. Detect Robust Features

Summary:

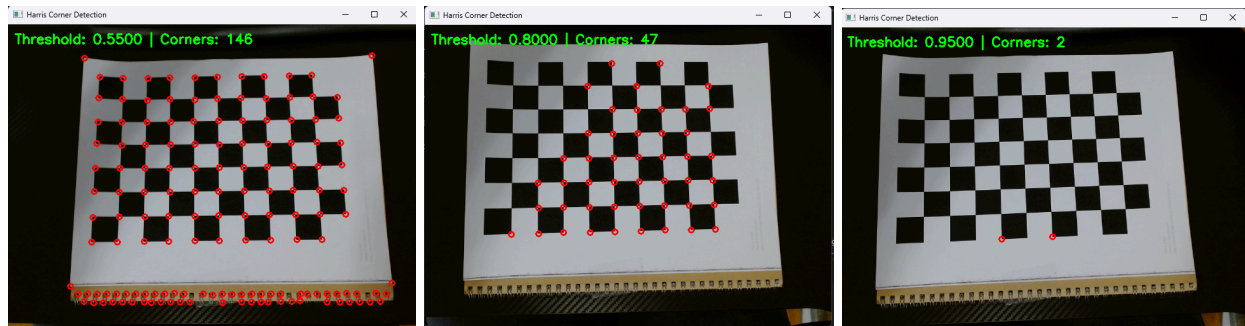
A standalone program detects and visualizes Harris corners in a live webcam feed in real time. The user can adjust the detection threshold interactively using `+` and `-` keypresses to observe how threshold affects the number and quality of detected corners.

Each frame is converted to grayscale and passed to `cv::cornerHarris()`, which produces a per-pixel response map encoding how corner-like each location is. The response is normalized to 0-255, then temporally smoothed using `cv::addWeighted()` across frames to reduce flicker. A pixel is marked as a corner only if it exceeds the threshold AND is the local maximum in its 10x10 neighborhood (non-maximum suppression). Detected corners are drawn as red circles on the live frame.

Key decisions:

- *Threshold relative to global max ($val > threshold * global_max$) rather than a fixed value:* ensures the threshold behaves consistently regardless of image content or lighting conditions.
- *Temporal smoothing (0.8/0.2 weighted average):* blends 80% of the accumulated response with 20% of the current frame, stabilizing detections against frame-to-frame noise without making the system slow to respond to actual movement.
- *Non-maximum suppression with 10x10 window:* prevents clusters of nearby pixels all firing simultaneously, ensuring each detected corner represents one distinct structural point.
- *Single file (`features.cpp`):* Task 7 is independent of all previous calibration and AR code, so a single self-contained file is cleaner than extending the existing project structure.

Demo Images:



Checkerboard results (Images 1, 2, 3):

- **Image 3 (threshold 0.55, 146 corners):** At a lower threshold, Harris detects corners at nearly every internal checkerboard intersection plus the outer board edges and the spiral binding of the notebook below. The corners are densely distributed across the entire pattern, including weaker detections along the board boundary.
- **Image 1 (threshold 0.80, 47 corners):** At a medium threshold, the number of detected corners drops significantly to 47 - only the strongest, most geometrically distinct internal intersections survive. The detections are visibly cleaner and more accurately centered on the true corner locations.
- **Image 2 (threshold 0.95, 2 corners):** At a very high threshold, only the 2 globally strongest corners in the entire frame survive, located near the bottom edge of the board where the contrast between the black squares and white paper is sharpest. This demonstrates that the threshold directly controls the minimum corner strength required for detection.



Natural scene results (Images 4, 5, 6 - Silicon Valley poster):

- **Image 5 (threshold 0.65, 133 corners):** At a medium-low threshold, Harris detects 133 corners distributed across the entire poster- concentrated at building edges, text letter corners, logo boundaries, and the circular balloon/sphere outlines. This confirms Harris responds to genuine geometric structure in natural images, not just synthetic patterns.

- **Image 6 (threshold 0.80, 25 corners):** Raising the threshold to 0.80 reduces detections to 25, the surviving corners cluster around the most distinctive structural features: sharp building corners, high-contrast text edges, and the circular Uber balloon. These are the most repeatable and reliable points in the scene.
- **Image 4 (threshold 0.99, 1 corner):** At the maximum threshold only a single globally dominant corner survives, located at the bottom-left edge of the poster where the paper meets the dark background, the highest contrast boundary in the entire scene.

Cross-scene Observations:

- The checkerboard produces more evenly distributed corner responses than the natural scene because its pattern has consistent, high-contrast intersections across the entire image. The poster produces fewer but more spatially clustered corners because its distinctive features are concentrated at specific structural locations.
- Temporal smoothing (**addWeighted** with 0.8/0.2 ratio) significantly stabilized the detections- the same threshold value now produces a consistent corner count across frames rather than flickering between widely different values.
- The threshold's effect is non-linear- small increases near the top of the range (0.90 to 0.99) eliminate corners far more aggressively than the same step size near the bottom (0.55 to 0.65), reflecting the non-uniform distribution of corner response magnitudes in the scene.

How Harris corners could enable AR without a checkerboard?

The checkerboard-based AR pipeline in Tasks 1 to 6 works because every corner has a known, fixed 3D world coordinate, you define them yourself as $(0,0,0)$, $(1,0,0)$, etc. This known correspondence between 2D image points and 3D world points is what makes `solvePnP` work.

Feature-based AR replaces the checkerboard with this pipeline:

Step 1 - Offline: build a reference map.

Point the camera at your target object (a book cover, painting, or any textured surface) and detect Harris corners. For each corner, record its 2D position in the reference image. Then, either manually measure or reconstruct (via stereo or structure-from-motion) the corresponding 3D world coordinates of those points on the object's surface.

Step 2 - Online: detect and match.

For each new camera frame, detect Harris corners again. Match each detected corner to the reference map using a descriptor (Harris alone gives location but not a descriptor- in practice you would pair it with a descriptor like BRIEF or use a full feature detector like SIFT or ORB that gives both location and description). Each successful match gives you one 2D-to-3D correspondence.

Step 3 - Estimate pose.

Pass the matched 2D-3D pairs into `solvePnP` exactly as in Task 4. This gives you `rvec` and `tvec` for the target object in the current frame.

Step 4 - Render.

Use `projectPoints` exactly as in Tasks 5 and 6 to draw virtual objects relative to the target- same pipeline, different source of correspondences.

The key limitation is that Harris corners alone are not sufficient for reliable matching because they provide no descriptor: two corners at similar locations in two different frames cannot be distinguished from each other by position alone.

This is why real feature-based AR systems use SIFT, SURF, or ORB, which attach a robust descriptor to each detected point. Harris is the conceptual foundation that these methods build on.

Extension #1

Harris vs ORB: A Comparative Investigation of Feature Detection Techniques

1. Motivation

Classical AR pipelines like the one implemented in Tasks 1 through 6 of this project rely on structured calibration targets (checkerboards) to provide known 2D-to-3D point correspondences for pose estimation. While effective, this approach has a fundamental limitation: it requires the target to be present in every frame. Real-world AR applications need to work on arbitrary scenes without predefined markers.

Feature detection addresses this by automatically identifying distinctive, repeatable points in any image. However, not all feature detectors are equal. This investigation compares two fundamentally different approaches:

- *Harris Corner Detection*: a classical gradient-based corner detector with no descriptor
- *ORB (Oriented FAST and Rotated BRIEF)*: a modern, descriptor-equipped feature detector designed for real-world matching

The central question is: how do these two approaches behave differently on structured vs natural scenes, and what does that imply for real-world AR use cases?

2. Background

Harris Corner Detection

Harris detects corners by analyzing the local intensity gradient structure around each pixel. A pixel is considered a corner if intensity changes significantly in all directions, as opposed to an edge (change in one direction only) or a flat region (no change). This is quantified through the Harris response matrix, which produces a per-pixel score. Pixels with a high score are corner candidates. A threshold is applied to filter weak responses, and non-maximum suppression ensures only local peak responses are retained. Harris provides only pixel locations, with no information about the appearance of the surrounding region.

ORB Feature Detection

ORB combines two existing techniques: FAST for keypoint detection and BRIEF for descriptor computation. Unlike Harris, ORB assigns each detected keypoint a 32-byte binary descriptor encoding local pixel intensity comparisons in a patch around the point. This descriptor enables

matching: given two images, ORB features can be compared using Hamming distance to find corresponding points. ORB is also scale and rotation invariant, meaning it can find the same feature even if the object is viewed from a different distance or angle.

3. Experimental Setup

Implementation

Both detectors were implemented in a single real-time program (`features_compare.cpp`) operating on a live webcam feed. Key implementation decisions:

Harris implementation:

- `blockSize=2, apertureSize=3, k=0.04` (standard OpenCV defaults for the corner response matrix)
- `cv::cornerHarris()` produces a raw float response map where each pixel's value represents its corner-ness score
- Response normalized to 0-255 using `cv::normalize()` with `NORM_MINMAX` to ensure consistent thresholding across varying lighting conditions
- Threshold applied as a fraction of the global maximum response per frame: `val > threshold * global_max`. This relative thresholding ensures the detector always finds the strongest corners in the scene regardless of absolute intensity values
- Temporal smoothing applied using `cv::addWeighted(accumulated, 0.8, raw_response, 0.2, 0, accumulated)` to reduce frame-to-frame flicker caused by camera noise
- Non-maximum suppression applied in a 10x10 neighborhood using `cv::minMaxLoc()` on each candidate's surrounding patch, ensuring only the single strongest response in each local region is retained as a corner

ORB implementation:

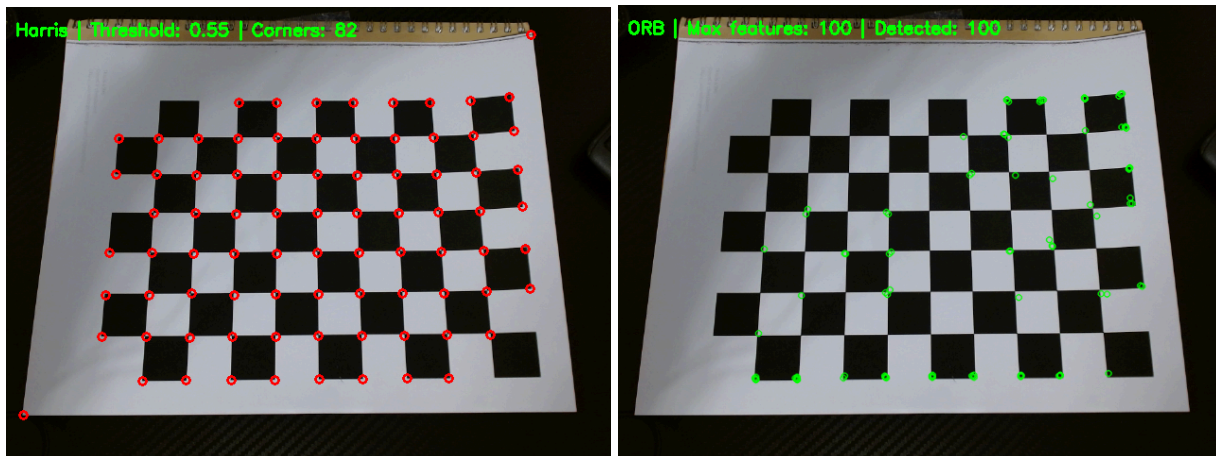
- `cv::ORB::create(nfeatures=100)` initializes the detector with a maximum feature count of 100
- `orb->detectAndCompute(gray, cv::noArray(), keypoints, descriptors)` performs detection and description in a single call, returning both keypoint locations and their corresponding 32-byte binary descriptors
- Each descriptor encodes 256 binary comparisons between pixel pairs in a patch around the keypoint, computed after rotating the patch to align with the keypoint's dominant orientation (ensuring rotation invariance)
- Keypoints are drawn using `cv::DrawMatchesFlags::DEFAULT` which renders plain circles, allowing direct visual comparison with Harris detections

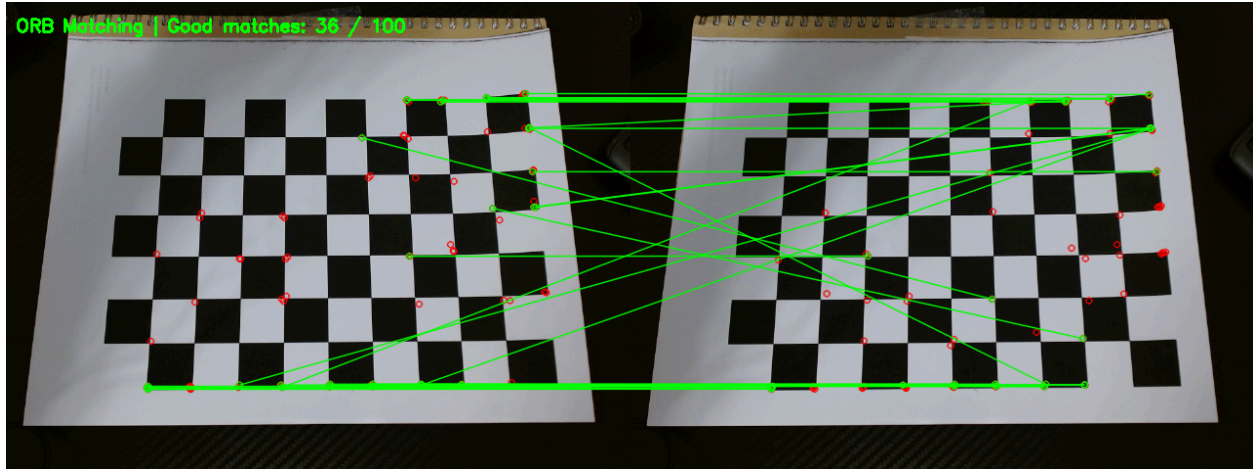
Matching pipeline:

- A `cv::BFMatcher` (Brute Force Matcher) with `cv::NORM_HAMMING` distance metric is used. `NORM_HAMMING` counts the number of differing bits between two binary descriptors and is the correct distance metric for ORB's binary descriptor format
- Each frame's descriptors are matched against the previous frame's descriptors using `matcher.match(descriptors, prev_descriptors, matches)`, producing one match per current-frame feature
- Matches are sorted in ascending order of distance: lower distance indicates higher descriptor similarity and therefore higher match confidence
- Match quality filtering uses a dynamic distance threshold rather than a fixed count: only matches satisfying `m.distance <= 3.0 * min_dist` are retained, where `min_dist` is the distance of the single best match in the current frame. This ensures the good match count genuinely reflects correspondence quality. When features are ambiguous (as on the checkerboard), `min_dist` is low but so are many other distances, causing the threshold to be overly permissive. When features are distinctive (as on the poster), `min_dist` is low while most other distances are significantly higher, allowing clean separation of good and bad matches
- Retained matches are visualized using `cv::drawMatches()`, drawing green lines connecting each matched feature pair across the two frames. The spatial pattern of these lines (parallel vs criss-crossed) is the primary diagnostic for match quality

4. Results:

A) Structured Target (Checkerboard)





Harris detection:

- 82 corners detected at threshold 0.55
- Detections are densely and uniformly distributed across all internal intersections of the checkerboard
- Additional detections present along the outer board boundary where the paper meets the dark background
- Harris correctly identifies every high-contrast corner in the structured pattern

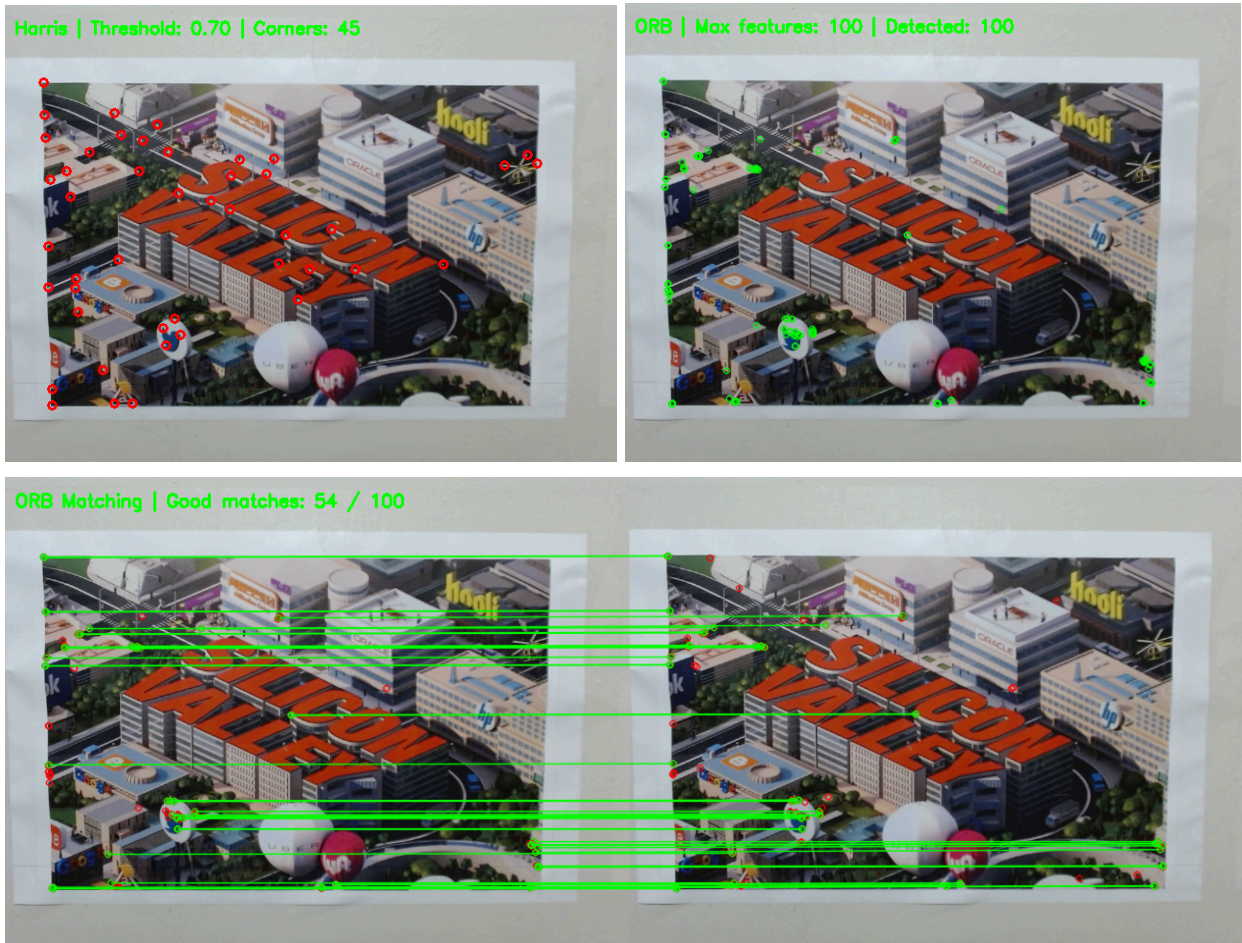
ORB detection:

- 100 features detected (at maximum feature limit)
- Detections are concentrated heavily along the outer boundary corners of the board
- Very few internal intersections are detected despite being geometrically valid corners
- ORB deprioritizes internal intersections because every internal corner's local patch looks nearly identical to every other, resulting in low uniqueness scores

ORB matching:

- 36 good matches out of 100 retained after distance threshold filtering
- Matching lines are heavily criss-crossed and non-parallel, with lines connecting points on one row to points on a different row in the adjacent frame
- The chaotic line pattern confirms that the matcher frequently confuses one checkerboard corner for another due to descriptor similarity
- Match quality is fundamentally limited by the repetitive nature of the pattern rather than by the matching algorithm itself

B) Natural Scene (Silicon Valley Poster)



Harris detection:

- 85 corners detected at threshold 0.55
- Detections are distributed across building edges, text letter corners, logo boundaries, road intersections, and the poster boundary
- Distribution is reasonably uniform but includes boundary detections where the paper meets the wall background
- Harris correctly identifies geometrically strong corners throughout the scene

ORB detection:

- 100 features detected (at maximum feature limit)
- Detections spread across the interior of the poster: building rooftops, text edges, circular logo boundaries, street intersections, and balloon outlines
- Notably fewer detections along the poster paper boundary compared to Harris, as the straight paper edge lacks the local uniqueness required for a high ORB score

- Feature distribution reflects distinctiveness rather than pure geometric corner-ness

ORB matching:

- 54 good matches out of 100 retained after distance threshold filtering
- Matching lines are predominantly parallel and horizontal, correctly connecting the same physical features across both frames
- Building corners, text edges, and logo boundaries are matched to their correct corresponding locations with minimal cross-matching errors
- The quality and consistency of matches on the poster stands in direct contrast to the checkerboard results

5. Comparative Analysis

The contrast in checkerboard detection reveals the core philosophical difference between the two approaches. Harris asks "is this a corner?" ORB asks "is this corner unique enough to find again in a different image?" For the checkerboard, the answers diverge dramatically. For the poster, they converge.

A) Matching quality

The matching results provide the clearest evidence of the practical difference between the two detectors:

- On the checkerboard, 36 good matches were retained but the line pattern was chaotic, indicating many of those 36 matches connect incorrect physical correspondences that happened to have similar descriptor distances
- On the poster, 54 good matches were retained with predominantly parallel, geometrically consistent lines, indicating the majority of matches correctly identify the same physical point across frames

The good match count alone (36 vs 54) understates the quality difference. The spatial coherence of the match lines is the more meaningful metric. Parallel lines indicate geometrically consistent matches. Criss-crossed lines indicate ambiguous matches where the descriptor distance happened to be low but the physical correspondence is wrong.

B) Descriptor uniqueness as the determining factor

The performance gap across both targets is entirely explained by descriptor uniqueness. On the poster, each feature's 32-byte descriptor encodes a local patch that looks different from every other patch in the image. The matcher can confidently identify the closest match. On the checkerboard, descriptors for different corners are nearly identical because the underlying

patches are nearly identical. The matcher has no reliable basis for distinguishing one corner from another, producing correct-looking distance scores for incorrect matches.

This is quantified by the minimum match distance behavior: on the checkerboard, the minimum distance is very low but so are most other distances, making the 3x threshold overly permissive. On the poster, the minimum distance is also low but other distances are meaningfully higher, allowing the threshold to correctly separate good matches from bad ones.

6. Real-World Implications

Why Harris is insufficient for AR

Harris detects corners reliably and efficiently but provides no mechanism for identifying the same corner across frames. Without a descriptor, there is no way to answer the question "is this the same physical point I saw in the last frame?" This makes Harris unsuitable as a standalone basis for any AR pipeline requiring frame-to-frame feature tracking or matching against a reference image.

Why ORB enables markerless AR

The matching results on the poster demonstrate the core capability that makes markerless AR possible. ORB reliably finds the same physical points across consecutive frames on a natural scene, providing 2D-to-2D correspondences. Combined with RANSAC-based outlier filtering and a homography or `solvePnP` pose estimation step, these correspondences can estimate how a target has moved between frames. The intrinsic calibration from the checkerboard pipeline (Tasks 1 to 3) can then be reused directly, meaning the entire AR rendering pipeline from Tasks 4 to 6 could be applied to any textured target without modification, simply by replacing `findChessboardCorners` with ORB-matched correspondences.

Where each technique is appropriate

- *Harris*: structured calibration pipelines, educational demonstrations, and preprocessing where descriptors are handled separately. The checkerboard pipeline in this project implicitly uses a Harris-based approach inside `findChessboardCorners`
- *ORB*: real-world feature matching, markerless AR, object recognition, and any application requiring reliable cross-frame correspondence on natural scenes. ORB is used in production AR systems including early versions of ARCore
- *SIFT/SURF*: more accurate than ORB at the cost of speed and patent restrictions. Appropriate when matching quality is more critical than real-time performance
- *Deep learning features (SuperPoint, D2-Net)*: current state of the art, learning descriptors from data rather than engineering them, outperforming classical methods on challenging scenes

Extension #2

Multiple Targets Simultaneously

Summary:

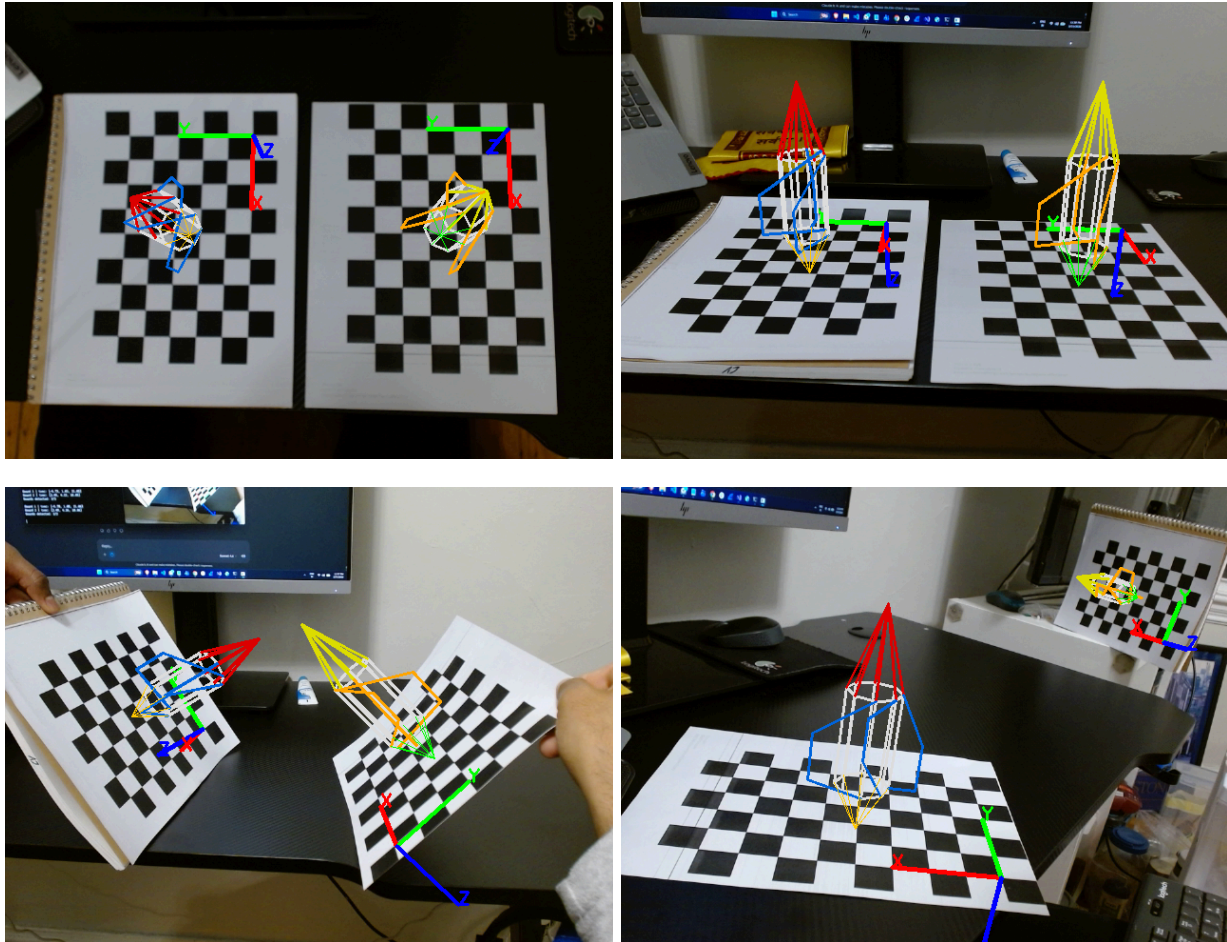
The AR system is extended to detect and track two independent checkerboard targets simultaneously in a single frame, rendering a distinctly colored virtual rocket above each board with fully independent pose estimation.

Since `findChessboardCorners` finds only one board per call, a two-pass masking strategy is implemented. The first pass detects board 1 normally. Its corner bounding rectangle is computed, expanded by a margin, and filled with flat gray (128, 128, 128) on a copy of the frame- removing the contrast the detector relies on. The second pass runs on this masked frame, finding board 2 without interference. Each board then gets its own independent `solvePnP` call and rendering pipeline on the same output frame.

Key decisions:

- *Two-pass masking:*
cleanest solution to the single-board limitation without modifying detection logic or introducing additional libraries
- *Margin expansion on bounding box:*
detected corners only cover internal intersections, not the full board area. Without expansion, outer board squares remain unmasked and can retrigger detection in the second pass
- *Gray fill value (128):*
avoids creating new high-contrast edges at the mask boundary that could themselves be detected as corners
- *Color scheme parameter added to `drawVirtualObject`:*
single integer parameter selects between two color palettes rather than duplicating the entire function
- *Separate `rvec` and `tvec` per board:*
ensures pose estimates are completely decoupled with zero cross-interference between boards

Demo Images:



Observations:

- *Image 1 (both boards flat, top-down view):*
Both rockets appear compressed into near-flat cross-sections because the camera looks straight down at both boards, nose points directly toward the camera from both. The coordinate axes of both boards are nearly parallel, confirming similar physical orientations. `tvec[2]` is comparable for both boards as expected at similar depths.
- *Image 2 (both boards flat, camera in front):*
The clearest result. Both rockets stand fully upright simultaneously with all components visible. Coordinate axes are nearly parallel between boards, and `tvec` values differ only in X (horizontal separation), with Y and Z matching closely. This is physically correct for two side-by-side boards at the same depth.

- *Image 3 (boards tilted in opposite directions):*
The strongest demonstration of independent tracking. Board 1 tilted left produces a left-leaning rocket, board 2 tilted right produces a right-leaning rocket. The Z axes of the two boards visibly diverge in opposite directions, directly encoding the opposite tilts in `rvec`. Neither board's pose is affected by the other's orientation.
 - *Image 4 (one board near, one far):*
Board 1 close up renders a large detailed rocket; board 2 far away renders a correctly scaled-down version of the same rocket. `tvec[2]` differs significantly between boards, and `projectPoints` scales both rockets proportionally with depth, exactly as a real 3D object would behave.
-

Reflections:

- Camera calibration quality depends far more on the variety of board poses captured than on the number of frames. This was not obvious going in and became clear only after seeing how different viewing angles contribute independent constraints to the solver.
 - The relationship between 2D image points and 3D world coordinates is the conceptual backbone of the entire project. Once that pairing clicked, everything from calibration through to AR rendering felt like a natural extension of the same idea.
 - Seeing `solvePnP` respond correctly to physical board movement in real time made the abstract pose estimation math feel concrete and intuitive in a way that reading about it does not.
 - The Harris vs ORB comparison was the most conceptually rich part of the project. The difference between detecting corners and detecting matchable features is subtle on paper but immediately obvious when you see ORB correctly tracking a natural scene while Harris has no equivalent capability.
 - The multiple target extension revealed that a well-designed modular pipeline scales naturally. Running the same pipeline twice with independent inputs required minimal additional code, which reflects good architectural decisions made early in the project.
 - The checkerboard exposed an interesting irony: the regularity that makes it an ideal calibration target makes it a poor feature matching target, because every corner's local appearance is identical to every other.
-

Acknowledgement:

- I would like to express my sincere gratitude to **Professor Bruce Maxwell** for his exceptional teaching and patience in addressing my most fundamental questions, explaining concepts multiple times until they clicked.
 - I am grateful to the **course teaching assistants** for their consistent support and readiness to help whenever I encountered difficulties. Their guidance on debugging issues and clarifying implementation details was invaluable throughout the project development process.
 - The **OpenCV documentation** served as an essential reference throughout this project, particularly for understanding the parameters and behavior of calibration functions, pose estimation techniques, coordinate system conventions, and feature detection methods including Harris and ORB.
 - **YouTube tutorials** provided supplementary visual explanations that complemented course materials, particularly helpful for building intuition around camera calibration geometry, the relationship between rotation and translation vectors in 3D space, and the conceptual differences between classical feature detectors.
 - Special thanks to **Claude** for being available 24/7 to help me ideate, refine half-formed concepts, explore implementation possibilities, understand new techniques, and provide subject matter expertise that accelerated my learning process throughout this project.
-